

FLIR Mobile SDK migration guide

Migration from FLIR One iOS SDK (versions 3.+)

Table of Content

[TOC]

Introduction

Welcome to the developers guide for migrating legacy FLIR One SDK for iOS (versions 3+) Apps to the powerful and feature rich FLIR Mobile SDK. This document will introduce you to developing apps with FLIR One external iPhone accessories (aka the FLIR One device) using FLIR Mobile SDK (a successor to FLIR One SDK for iOS).

This document intends to serve as a reference for developers experienced with FLIR One SDK for iOS. API mappings between SDKs will be presented in the form of a lookup table with code examples for FLIR Mobile SDK. This guide provides you with a summary and a detailed list of changes, updates and new APIs available in FLIR Mobile SDK. Applications written for FLIR One iOS SDK require adjustments to implement the FLIR Mobile SDK and this document will guide you through the changes.

Note: the FLIR Mobile SDK offers a wide feature set, with many functions that were not available in the FLIR One SDK for iOS. This document describes only part of the available API. Refer to FLIR Mobile SDK official documentation and samples for a full feature overview.

In the following sections, you will read about features supported by FLIR One SDK for iOS (version 3.+) and how to implement them using FLIR Mobile SDK.

Conform to delegates

FLIR One SDK

- FLIROneSDKImageEditorDelegate
- FLIROneSDKImagePropertyController
- FLIROneSDKImageReceiverDelegate
- FLIROneSDKStreamManagerDelegate
- FLIROneSDKVideoRendererDelegate

FLIR Mobile SDK

- FLIRDataReceivedDelegate
- FLIRDiscoveryEventDelegate
- FLIRCameraImportEventDelegate
- FLIRRemoteDelegate

Additional comments

To start using the new FLIR Mobile SDK, you need to conform with the *FLIRDataReceivedDelegate* and *FLIRDiscoveryEventDelegate* protocols. When a camera is connected, the protocols *FLIRCameraImportEventDelegate* and *FLIRRemoteDelegate* can be extended to provide additional events.

Device discovery

FLIR One SDK

- N/A

FLIR Mobile SDK

- [discovery start:FLIRCommunicationInterfaceLightning]
- [discovery stop]
- [discovery isDiscovering]

Additional comments

The *FLIRCommunicationInterface* enum represents the possible camera interfaces. In this release, only the network, emulator and lightning enums are implemented.

Once a device is found, the *cameraFound* delegate method is called and it’s possible to handle the discovered camera identity object.

:warning: Legacy FLIR One SDK for iOS did not implement discovery methods.

Code example (using FLIR Mobile SDK)

```
- (IBAction)connectDevice:(id)sender {
    [discovery start:FLIRCommunicationInterfaceLightning];
}

- (void)cameraFound:(nonnull FLIRIdentity *)cameraIdentity {}

- (void)discoveryError:(nonnull NSString *)error netServiceError:(int)nsnetserviceserror on:(FLIRCommunicationInterface)iface {}
```

Connect to device

FLIR One SDK

- FLIROneSDKDidConnect
- FLIROneSDKDidDisconnect

FLIR Mobile SDK

- [camera connect:cameraIdentity]
- [camera disconnect]

Additional comments

Legacy FLIR One SDK for iOS triggered the connection just after the FLIR ONE device had established communication without any scanning method. The new FLIR Mobile SDK first finds the device, then a connect method needs to be called.

The disconnect method in the legacy FLIR One SDK for iOS was triggered just after the FLIR ONE device has lost communication. In the new FLIR Mobile SDK, the disconnect method can be programmatically called.

Connecting to a device does not start live streaming automatically.

Code example (using FLIR Mobile SDK)

```
- (void)cameraFound:(nonnull FLIRIdentity *)cameraIdentity {  
  
    if([cameraIdentity cameraType] == FLIRCameraTypeFlirOne)  
    {  
        [camera connect:cameraIdentity];  
    }  
}
```

Stream frames

FLIR One SDK

- @interface FLIROneSDKStreamManager
- FLIROneSDKDidFinishCapturingPhoto:withFilepath:
- FLIROneSDKDidStartRecordingVideo:
- FLIROneSDKDidStopRecordingVideo:
- FLIROneSDKDidFinishWritingVideo:withFilepath:

FLIR Mobile SDK

- [camera subscribeStream]

Additional comments

In the legacy FLIR One SDK for iOS the *FLIROneSDKStreamManager* was used for controlling data capture from a live image stream. The *FLIROneSDKStreamManagerDelegate* implemented methods to respond to *FLIROneSDKStreamManager* and Capture Photos or Videos. In the new FLIR Mobile SDK, start grabbing frame events by implementing the *subscribeStream* method.

:warning: When the device connects, you need to subscribe the Camera to frame grabbing events.

Code example (using FLIR Mobile SDK)

```
- (void)onConnectionStatusChanged:(FLIRCamera *_Nonnull) camera Status:(FLIRConnectionStatus)status withError:(nullable NSError *)error {  
    if(status == FLIRConnectionStatusConnected)  
        [camera subscribeStream];  
  
    if(status == FLIRConnectionStatusDisconnected)  
        [camera unsubscribeStream];  
}
```

Render received frames

FLIR One SDK

- FLIROneSDKDelegateManager:didReceiveFrameWithOptions:metadata:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveBlendedMSXRGBA8888Image:imageSize:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveThermal14BitLinearFluxImage:imageSize:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveThermalRGBA8888Image:imageSize:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveVisualJPEGImage:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveVisualYCbCr888Image:imageSize:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveRadiometricData:imageSize:sequenceNumber:
- FLIROneSDKDelegateManager:didReceiveRadiometricDataFloat:imageSize:sequenceNumber:

FLIR Mobile SDK

- imageReceived

Additional comments

Legacy FLIR One SDK for iOS the *FLIROneSDKImageReceiverDelegate* allowed users to receive live frames from the camera in the image formats of their choice, or to load received images generated from a file on disk in the same formats. In the new FLIR Mobile SDK, the method *imageReceived* is raised when an image is received from the camera. You need to use *withImage* to ensure thread-safe access to the received *FLIRThermalImage*.

:no_entry: Missing *glkView*. Missing *glkViewImageOptions*.

:ballot_box_with_check: The legacy FLIR One SDK for iOS you could optionally render your thermal image on a *glkView*. Using the new FLIR Mobile SDK, your application is provided a rendered image which you can then display or process as needed in your App.

Code example (using FLIR Mobile SDK)

```
- (void) imageReceived {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->camera withImage:^(FLIRThermalImage *image) {
            self->imageview.image = [image getImage];
        }];
    });
}
```

Get and set parameters for incoming frame (i.e. palette, rotation, emissivity)

FLIR One SDK

- FLIROneSDKPalette *palette = [[[FLIROneSDKPalette palettes] allValues] objectAtIndex:0];
- [[FLIROneSDKStreamManager sharedInstance] setPalette:palette];

FLIR Mobile SDK

- [FLIRPaletteManager getDefaultPalettes]
- FLIRThermalImage().palette = FLIRThermalImage().paletteManager.arctic

Additional comments

To access the palette objects in the new FLIR Mobile SDK, use *FLIRPALETTEManager* instead of accessing palettes in the class *FLIROneSDKPalette*, as was the case in the legacy FLIR One SDK for iOS.

Instead of using the deprecated *FLIROneSDKImageMetadata* to access to the thermal image’s properties, emissivity and palette, use the *FLIRImageParameters* to get or set the default emissivity and use *FLIRThermalImage* to get or set palette properties of the *ThermalImage*.

Code example (using FLIR Mobile SDK)

```
- (void) imageReceived {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->camera withImage:^(FLIRThermalImage *image) {
            self->imageView.image = [image getImage];
            FLIRImageParameters *params = [image getImageParameters];
            float emissivityValue = params.emissivity;
            image.Palette = image.PaletteManager.iron;
            params.emissivity = (0.91);
            [image setRotate:ROTATION_ANGLE_90_DEGREES];
        }];
    });
}
```

Read image information (i.e width)

FLIR One SDK

- [FLIROneSDKUIImage imageWithFormat:FLIROneSDKImageOptionsThermalRadiometricKelvinImageFloat andData:radiometricData andSize:size]

FLIR Mobile SDK

- image.getWidth;

Additional comments

The new FLIR Mobile SDK adds a feature to easily access the image information such as width and height using the *FLIRImageBase* class. In the legacy FLIR One SDK for iOS, access to the size of the image was available using the *imageWithFormat* callback.

:warning: *FLIROneSDKImageOptions* was replaced by *FUSION_MODE*.

Code example (using FLIR Mobile SDK)

```
- (void) imageReceived {
    dispatch_async(dispatch_get_main_queue(), ^{
        [self->camera withImage:^(FLIRThermalImage *image) {
            CGFloat imageWidth = image.getWidth;
        }];
    });
}
```

Save image

FLIR One SDK

- finishEditingAndSaveCurrentImageToFilepath:(NSURL)*filepath withPreviewImage:(UIImage)*previewImage;
- capturePhotoWithFilepath:(NSURL *)filepath

FLIR Mobile SDK

- saveAs: (NSString *) fileName;
- saveAs: (NSString) *fileName imageWithOverlay:(UIImage)* imageWithOverlay;
- saveAs: (NSString) *fileName imageViewWithOverlay:(UIImageView)* overlay;

Additional comments

In the new FLIR Mobile SDK, it is possible to save an image with overlay as a *UIImage* or *UIImageView*.

Code example (using FLIR Mobile SDK)

```
- (void) imageReceived {
    dispatch_async(dispatch_get_main_queue(), ^{
        UIImage *uiImage;
        [self->camera withImage:^(FLIRThermalImage *image) {
            [image saveAs:@"thermalImage"];
            [image saveAs:@"thermalImage" imageViewWithOverlay:imageview];
            [image saveAs:@"thermalImage" imageWithOverlay:uiImage];
        }]];
    });
}
```

Tuning/NUC

FLIR One SDK

Delegate Methods: - setAutomaticTuning - performTuning - FLIROneSDKTuningStateDidChange: - FLIROneSDKAutomaticTuningDidChange:

FLIR Mobile SDK

Interface FLIRCalibration - isActive - performNUC - subscribeCalibrationState - unsubscribeCalibrationState

Interface FLIRRemoteControl - getCalibration()

Delegate Methods: - CalibrationStateChanged

Additional comments

In the legacy FLIR One SDK for iOS, the *FLIROneSDKStreamManagerDelegate* implemented the *FLIROneSDKTuningStateDidChange* and *FLIROneSDKAutomaticTuningDidChange* callback methods. In the new FLIR Mobile SDK, this is replaced by the *FLIRRemoteDelegate* which implements *CalibrationStateChanged*.

:no_entry: Missing the *FLIROneSDKTuningState* enum, therefore, we only get a boolean value if the state was changed.

:no_entry: Missing *setAutomaticTuning()*, automatic tuning is always enabled in the FLIR Mobile SDK

Code example (using FLIR Mobile SDK)

```
FLIRRemoteControl *remoteControl = [camera getRemoteControl];
FLIRCalibration *calibration = [remoteControl getCalibration];
//check if device supports RemoteControl
if(remoteControl != nil){
    //check if device supports Calibration
    if(calibration != nil){
        //Get calibration state
        bool isCalibrating = [calibration isActive];
        //Subscribes for camera NUC state notifications
        [calibration subscribeCalibrationState];
        [calibration performNUC]; //perform NUC
    }
}
```

Battery

FLIR One SDK

Delegate Methods - FLIROneSDKBatteryChargingStateDidChange - FLIROneSDKBatteryPercentageDidChange

FLIR Mobile SDK

Interface FLIRBattery - getChargingState - getPercentage - subscribeChargingState - subscribePercentage - unsubscribeChargingState - unsubscribePercentage

Delegate Methods - ChargingStateChanged - PercentageChanged

Additional comments

In the legacy FLIR One SDK for iOS, the *FLIROneSDKStreamManagerDelegate* implemented the *FLIROneSDKBatteryChargingStateDidChange* and *FLIROneSDKBatteryPercentageDidChange* callback methods. In the new FLIR Mobile SDK, this is replaced by the *FLIRRemoteDelegate* which implements *PercentageChanged* and *ChargingStateChanged*. In the new FLIR Mobile SDK, it is also possible to subscribe for remaining battery power and charging state notifications.

Code example (using FLIR Mobile SDK)

```
FLIRRemoteControl *remoteControl = [camera getRemoteControl];

//check if device supports RemoteControl
if(remoteControl != nil){
    FLIRBattery *battery = [remoteControl getBattery];
    if(battery != nil){ //check if device supports Battery
```

```
        //Get Battery Charging state
        FLIRChargingState state = [battery getChargingState];
        //Get Battery percentage
        int percentageLeft = [battery getPercentage];
        //Subscribes for battery charging state notifications.
        [battery subscribeChargingState];
        //Subscribes for remaining battery power notifications.
        [battery subscribePercentage];
    }
}
```

Camera name and serial number

FLIR One SDK

Interface FLIROneSDKStreamManager - isDongle - isGen3Dongle - isProDongle

FLIR Mobile SDK

Interface FLIRCameraInformation - modelName - name - displayName - description - serialNumber - osImagekitName - osImageKitName - swCombinationVersion - confKitName - article - date - width - height - firmwareRevision

Interface FLIRRemoteControl - getCameraInformation

Additional comments

The legacy FLIR One SDK for iOS verifies if the model connected is Dongle, Gen3 or Pro and returns a bool. The new FLIR Mobile SDK offers the ability to access the firmware revision, the FLIR Imager’s date, etc.

Code example (using FLIR Mobile SDK)

```
func cameraFound(_ cameraIdentity: FLIRIdentity) {
    var remoteControl: FLIRRemoteControl
    var cameraInformation: FLIRCameraInformation
    if(cameraIdentity.cameraType() == FLIRCameraType.flirOne){
        FLIRCamera().connect(cameraIdentity)
        remoteControl = FLIRCamera().getRemoteControl()!
        try! cameraInformation = remoteControl.getCameraInformation()
        cameraInformation.name()
        cameraInformation.modelName()
        cameraInformation.serialNumber()
    }
}
```

Camera emulator

FLIR One SDK

- FLIROneSDKSimulation class

FLIR Mobile SDK

- [discovery start:FLIRCommunicationInterfaceEmulator];

Additional comments

:no_entry: Missing FLIROneSDKSimulation class and its methods, however, it’s possible to initiate an interface emulator using the FLIRCommunicationInterfaceEmulator.

Manipulate Thermal Images created from file or stream

FLIR One SDK

- FLIROneSDKImageEditor class

FLIR Mobile SDK

- FLIRThermalImage

Additional comments

:ballot_box_with_check: In the legacy FLIR One SDK for iOS, the *FLIROneSDKImageEditor* was responsible for *load, manipulate, display* and *save* captured images from a live image stream. In the new FLIR Mobile SDK, the interface *FLIRThermalImage* defines features of a thermal image with both saved images and images from a live stream.

Recording videos

FLIR One SDK

- FLIROneSDKDidStartRecordingVideo
- FLIROneSDKDidStopRecordingVideo
- FLIROneSDKDidFinishWritingVideo:withFilepath
- startRecordingVideoWithFilepath:withVideoRendererDelegate:
- stopRecordingVideo
- FLIROneSDKVideoRendererDelegate

FLIR Mobile SDK

- N/A

Additional comments

:no_entry: Missing videos related methods and classes — the new FLIR Mobile SDK does not implement video saving functionality.

Additional features

Network settings

The new FLIR Mobile SDK brings a wide range of network capabilities.

FLIRAdapterInfo

Describes the network interface card that discovered a camera - name - isWireless - macAddress - ipAddress - mask

FLIRCameraDeviceInfo

Camera device information - @property ipAddress

FLIRCameraEvent

Handles events from a remotely connected camera.

```
// Listen for events from the camera with ipAddress
- (instancetype)initWithCameraIpAddress:(NSString *)ipAddress
```

FLIRIpSettings

Stores the IP settings of a discovered network camera - hosttarget - ipAddress - macAddress - defaultGateway - subnetMask - isDhcpEnabled - isValid - adapterInfo

FLIRIdentity

Used when discovering and connecting to cameras. When a camera is found using the *CommunicationInterface::network*. - ipSettings

Import images from camera

The new FLIR Mobile SDK brings the *FLIRCameraImport* class which facilitates importing files from Camera.

- getImages
- startImport:withDestPath
- cancelImport
- delegate
- listImages

GPS and Compas information

The new FLIR Mobile SDK provides latitude, longitude and compass information that are embedded in the image.

- FLIRImageBase
 - getCompassInformation
 - getGpsInformation

Object Distance, Humidity, External Optics and Atmospheric Temperature

The new FLIR Mobile SDK provides image parameters that can be set or get using the *FLIRImageParameters* class.

- emissivity
- objectDistance
- reflectedTemperature
- atmosphericTemperature
- externalOpticsTemperature
- externalOpticsTransmission
- transmission
- relativeHumidity

Measurements areas

The new FLIR Mobile SDK provides various classes that can help the developer analyze image statistics depending on the shape of the area. You can choose between circle, line, rectangle and it also provides a base class for other measurement shapes. The values in measurement spots can also be determined. - FLIRMeasurementArea - FLIRMeasurementCircle - FLIRMeasurementCollection - FLIRMeasurementLine - FLIRMeasurementMarker - FLIRMeasurementParameters - FLIRMeasurementRectangle - FLIRMeasurementShape - FLIRMeasurementSpot

Scale

In the new FLIR Mobile SDK it's possible to set a scale to define the way the palette is applied to a *ThermalImage*. - setRangeWithMin:andMax: - enableAutoAdjust - getRangeMin - getRangeMax - getImage - isAutoAdjustEnabled - setRangeMax - setRangeMin

Storage

In the new FLIR Mobile SDK is possible to manage stored images in the Camera. You can subscribe to be notified of the last stored image and also get the URL of the storage directory. - getImagesStorage - getLastStoredImage - subscribeLastStoredImage - unsubscribeLastStoredImage

More information

For more information, see the FLIR Mobile SDK official documentation. For any support questions, contact <https://flir.custhelp.com/>

This guide was brought to you by FLIR & Synaptic Technologies, Inc.